

A1: INIT (Initial Stage)

if it sees a Quote (")
switches to QS mode

(to ignore the string)

if it sees a letter
switches to ID mode

QS (Quoted String State)

It ignores everything, besides
closing Quote, then switches
to INIT

ID (Identifier State)

starts with a letter, then
adds letters / digits
ex. x2, count, End

=>

Finishes when it hits
a symbol like space,
+, ",", =

=>

Lastly it checks if its
a Keyword like, IF, INT,
Main

Code:

```

// Get the identifier from the file name
public static Set<String> getIdRegex(String filename) throws Exception {
    Set<String> identifiers = new HashSet();
    String line = null;

    // FIX 1: Added "STRING" to keywordsArray
    String keywordsArray = {"IF", "WRITE", "READ", "RETURN", "BEGIN", "END", "MAIN",
        "INT", "REAL", "STRING"};
    Set<String> keywords = new HashSet();
    for (String s : keywordsArray) {
        keywords.add(s);
    }

    StringReader reader = new StringReader(filename);
    BufferedReader br = new BufferedReader(reader);
    String line;
    while ((line = br.readLine()) != null) {
        code.append(line + "\n");
    }

    // FIX 2: Define the Regex Patterns
    // Identifier: Starts with a letter, followed by any number of letters or digits
    Pattern idPattern = Pattern.compile("[a-zA-Z][a-zA-Z0-9]*");
    // Quoted String: Starts with ", contains non-quotes, ends with "
    Pattern quotedStringPattern = Pattern.compile("\"[^\"]*\"");

    while ((line = br.readLine()) != null) {
        // FIX 3: Remove quoted strings first so their content isn't counted
        Matcher m_quotedString = quotedStringPattern.matcher(line);
        String lineWithoutQuotedStrings = m_quotedString.replaceAll("");

        Matcher m = idPattern.matcher(lineWithoutQuotedStrings);
        while (m.find()) {
            // FIX 4: Use m.group() instead of substring logic
            String id = m.group();

            if (!keywords.contains(id)) {
                identifiers.add(id);
            }
        }
    }
    return identifiers;
}

```

Read raw text and extract a list of unique variable names (identifiers), while ignoring other parts of code

A12: Rules

Array that holds forbidden words (IF, INT, STRING)

"[^"]*" = " String/symbols"

ex. "hello world", "test_V2"

"[a-zA-Z][a-zA-Z0-9]*" = " text/digits "

ex. "name", "test2"

Code:

```

public static Set<String> getIdRegex(String filename) throws Exception {
    // FIX 1: Added "STRING" to keywordsArray
    String keywordsArray = {"IF", "WRITE", "READ", "RETURN", "BEGIN", "END", "MAIN",
        "INT", "REAL", "STRING"};
    Set<String> keywords = new HashSet();
    for (String s : keywordsArray) {
        keywords.add(s);
    }

    FileReader reader = new FileReader(filename);
    BufferedReader br = new BufferedReader(reader);
    String line;

    // FIX 2: Define the Regex Patterns
    // Identifier: Starts with a letter, followed by any number of letters or digits
    Pattern idPattern = Pattern.compile("[a-zA-Z][a-zA-Z0-9]*");
    // Quoted String: Starts with ", contains non-quotes, ends with "
    Pattern quotedStringPattern = Pattern.compile("\"[^\"]*\"");

    while ((line = br.readLine()) != null) {
        // FIX 3: Remove quoted strings first so their content isn't counted
        Matcher m_quotedString = quotedStringPattern.matcher(line);
        String lineWithoutQuotedStrings = m_quotedString.replaceAll("");

        Matcher m = idPattern.matcher(lineWithoutQuotedStrings);
        while (m.find()) {
            // FIX 4: Use m.group() instead of substring logic
            String id = m.group();

            if (!keywords.contains(id)) {
                identifiers.add(id);
            }
        }
    }
    return identifiers;
}

```

Loop

m_quotedString.replaceAll("\"")

scans for data, deletes them so the identifier scanner doesn't think its a variable

strings

ex. x = "hello" + y => x = + y

idPattern.matcher(...)

after the scan it finds substring that matches the identifier rule

keywords.contains(id)

after it finds a match, it checks if its a Keyword, if not then it gets added to the list of identifier